F2 · SQL para análisis de datos

SELECT · JOIN · GROUP BY · CTE · Ventanas

AI Surfer · N1 Fundamentos

Por qué SQL

- Casi todo dato de empresa vive en una base relacional o en un almacén que se consulta en SQL
- Aparece en la gran mayoría de ofertas de datos, de analista a AI Engineer
- pandas, Spark y DuckDB hablan SQL o lo imitan

Regla: filtra y agrega en SQL, analiza en pandas

Mapa del módulo

1. Bases relacionales en 5 minutos
2. SELECT, WHERE, ORDER BY
3. Funciones y fechas
4. JOINs y el error de multiplicar filas
5. GROUP BY y HAVING
6. Subconsultas y CTE
7. Funciones de ventana

2.1 · Tablas que se relacionan por claves

```
clientes(cliente_id, nombre, ciudad)
pedidos(pedido_id, cliente_id, fecha)
lineas(pedido_id, producto_id, unidades, precio_unitario)
```

- Cada cliente aparece **una vez**; los pedidos apuntan a él
- Motores: SQLite (curso), PostgreSQL, DuckDB (analítica local)

2.2 · Leer filas

```sql
SELECT nombre, precio * stock AS valor_stock
FROM productos
WHERE categoria = 'Bebidas' AND precio < 2.0
ORDER BY precio DESC, nombre
LIMIT 10;
```

Orden mental: **FROM → WHERE → SELECT → ORDER BY → LIMIT**

Errores típicos de SELECT

- `= NULL` siempre es falso → usa **`IS NULL`**
- Textos con comillas **simples**
- `ORDER BY` sin segundo criterio no garantiza el orden en empates
- `DISTINCT` para quitar duplicados, no como parche de un JOIN mal hecho

2.3 · Funciones y fechas

Necesidad	SQLite	DuckDB / Postgres
Mes de una fecha	`strftime('%Y-%m', fecha)`	`date_trunc('month', fecha)`
Restar días	`date(fecha, '-7 days')`	`fecha - INTERVAL '7' day`
Rellenar nulos	`COALESCE(x, 0)`	`COALESCE(x, 0)`

Aprende el **patrón**; la sintaxis se consulta

2.4 · JOIN

Tipo	Devuelve
`INNER`	solo coincidencias
`LEFT`	toda la izquierda (+ nulos)
`FULL OUTER`	ambos lados

«¿Qué clientes no han comprado?» → `LEFT JOIN … WHERE p.pedido_id IS NULL`

El error de multiplicar filas

- `pedidos` (1 fila/pedido) ⋈ `lineas` (N filas/pedido) → **N filas por pedido**
- `SUM(pedidos.gastos_envio)` sale **N veces** mayor

Antídoto: `COUNT(*)` antes y después del JOIN; agrega en la tabla correcta o antes de unir

2.5 · De filas a resúmenes

```sql
SELECT categoria, COUNT(*) AS n, AVG(precio) AS precio_medio
FROM productos
GROUP BY categoria
HAVING COUNT(*) > 3;
```

- **WHERE** filtra filas *antes*; **HAVING** filtra grupos *después*
- Lo que no se agrega, va en el `GROUP BY`
- `SUM(CASE WHEN … THEN 1 ELSE 0 END)` = conteo condicional

2.6 · CTE: consultas en capas

```sql
WITH ventas_cliente AS (
  SELECT cliente_id, SUM(importe) AS total
  FROM pedidos GROUP BY cliente_id
)
SELECT c.nombre, v.total
FROM ventas_cliente v JOIN clientes c USING (cliente_id)
WHERE v.total > 1000;
```

Se lee de arriba abajo y se depura capa a capa

2.7 · Funciones de ventana

`GROUP BY` → **una fila por grupo** · `PARTITION BY` → **una métrica por fila**

Función	Para
`ROW_NUMBER()`	top-N por grupo
`RANK()`	ranking con empates
`SUM(x) OVER (PARTITION BY …)`	cuota sobre el total
`LAG(x)`	comparar con el mes anterior
`SUM(x) OVER (ORDER BY fecha)`	acumulado

Patrón top-N por grupo

```sql
WITH r AS (
  SELECT categoria, nombre, ingresos,
         ROW_NUMBER() OVER (PARTITION BY categoria
                            ORDER BY ingresos DESC) AS rn
  FROM ventas_producto
)
SELECT * FROM r WHERE rn <= 3;
```

Pregunta clásica de entrevista

Entregable

5 consultas sobre TiendaSur · resultado · **una frase para Dirección** por consulta

Filtra y agrega en SQL → analiza en pandas